

Chapter 2

Predicting Ad Clicks on a Social Media Platform

A visual, alternatives-driven ML system design case study

PRIMARY REFERENCE

Study foundation

This chapter began as personal study notes based largely on Machine Learning System Design Interview by Ali Aminian and Alex Xu. The restructuring, clarifications, trade-off discussions, diagrams, and interpretations are the author's own.

Chapter at a glance

Design area	Current direction
Objective	Predict calibrated click probability for ranking and revenue decisions
Product guardrail	Frequency policy plus hide and block behavior
Label strategy	Clicks are positive; unclicked impressions require an explicit policy
Baseline	Logistic Regression
Richer models	DCN and DeepFM as experiment directions
Large-scale architecture	Two-tower or ANN retrieval plus richer final CTR ranking
Validation	Shadow, canary, A/B testing, and calibration checks

1. Case map and minimum scope

Predict click probability for eligible ads and use that probability in ranking. Keep frequency control, calibration, and user-experience guardrails explicit.

Ad-click prediction design map

The chapter moves from product behavior and labels to ranking, calibration, and safe rollout.



Minimum-scope principle

Start with a calibrated Logistic Regression baseline. Add interaction modeling or retrieval complexity only when data and scale justify it.

Figure 2.1 - Ad-click prediction design map.

CONFIRMED INTERPRETATION
Binary label, probabilistic serving output
The training label is click or no click under the selected labeling policy, while the service consumes a probability used for ranking and revenue decisions.

OPEN DESIGN DISCUSSION
What is the minimum sufficient scope?
Main product action: _____
Eligible inventory boundary: _____
Latency assumption: _____
Explicitly out of scope: _____

2. Product behavior and repeated exposure

The model can rank the same ad repeatedly, but product policy should decide when repetition becomes fatigue.

Repeated exposure is a product decision

The model predicts click probability, but serving policy decides how often the same ad may reappear.

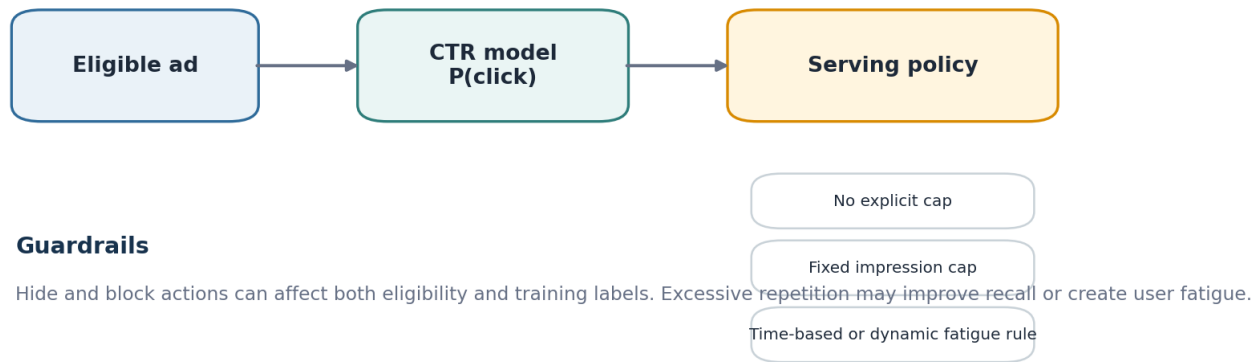


Figure 2.2 - Model prediction and repeated-exposure policy.

Policy option	Main benefit	Main cost
No explicit cap	Simplest serving logic; may improve recall	Can create fatigue or a poor user experience
Fixed impression cap	Transparent and easy to enforce	One rule may not fit every user, ad, or campaign
Time-based fatigue period	Prevents short-term repetition	Requires choosing an arbitrary time window
Dynamic rule	Can respond to engagement and hide behavior	More modeling and policy complexity

OPEN DESIGN DISCUSSION

Frequency cap or fatigue period?

Selected policy: _____

Evidence supporting it: _____

User-experience guardrail: _____

Condition that changes the policy: _____

3. Data, features, and uncertain negatives

Clicks provide clear positives. Viewed-but-unclicked impressions may be true negatives, missed opportunities, or unlabeled observations.

Data and label construction

Clicks are clear positives. Viewed-but-unclicked impressions are ambiguous.

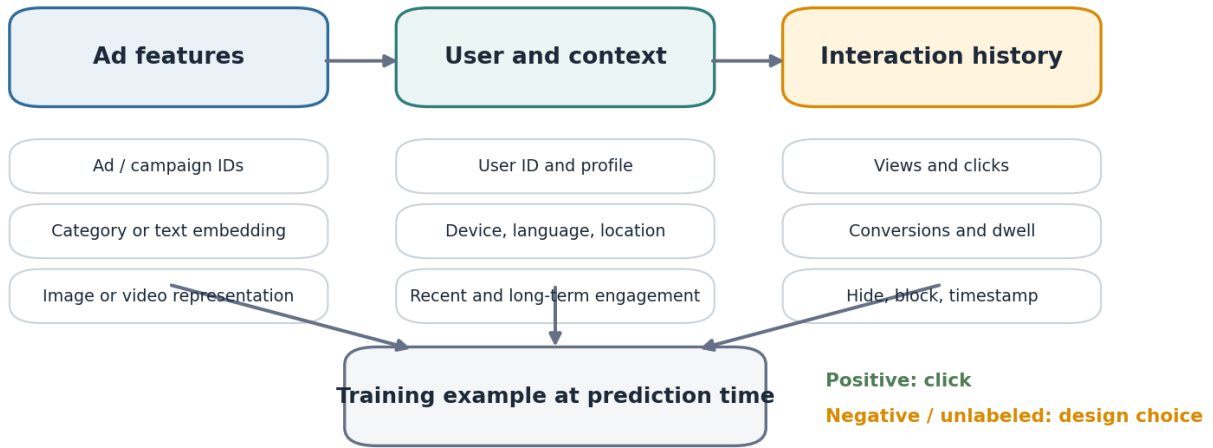


Figure 2.3 - Data families and label construction.

Labeling direction	Benefit	Risk
Every unclicked view is negative	Simple and scalable	Introduces label noise and hidden positives
Hide and block are stronger negatives	Uses explicit negative intent	Sparse and behavior-dependent
Unclicked views remain unlabeled	Represents uncertainty	Needs more complex estimation
Weighted or PU approach	May model hidden positives	Requires assumptions and class-prior estimation

STARTING RECOMMENDATION

Begin with a transparent labeling baseline

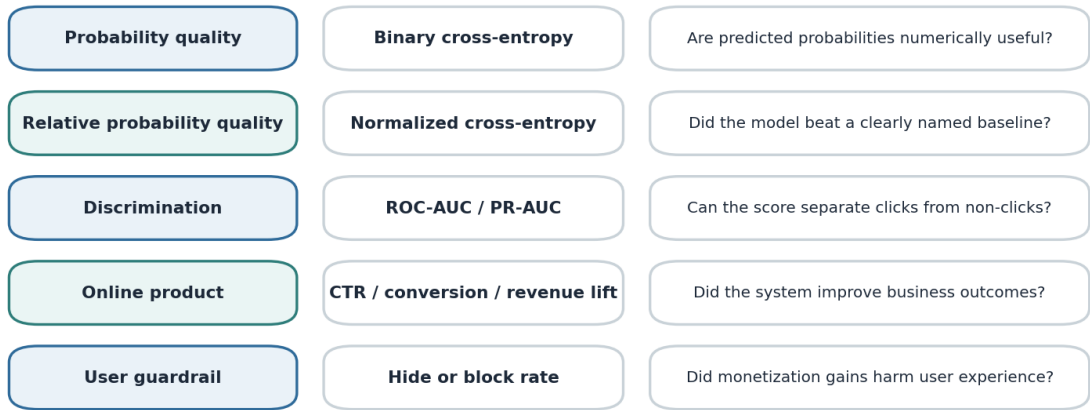
Use clicks as positives and a clearly defined impression-based negative policy as the first baseline. Compare weighted or PU-style methods only when analysis shows hidden positives materially limit quality.

4. Evaluation metrics and baselines

Use probability, discrimination, business, and user-experience metrics together. Name the baseline behind every normalized metric.

Metrics answer different questions

No single offline metric represents ranking quality, calibration, revenue, and user experience.



Normalized cross-entropy is interpretable only when the comparison baseline is explicit: constant historical CTR, current production model, or Logistic Regression.

Figure 2.4 - Metric families and their decisions.

Normalized CE baseline	Question answered	Limitation
Constant historical CTR	Did the model beat a naive prior?	Does not represent a learned production alternative
Current production model	Did the candidate improve over deployment?	Changes as production changes
Logistic Regression	Did richer modeling beat a simple learned baseline?	May not reflect current serving quality

OPEN DESIGN DISCUSSION

Which metric gates online testing?

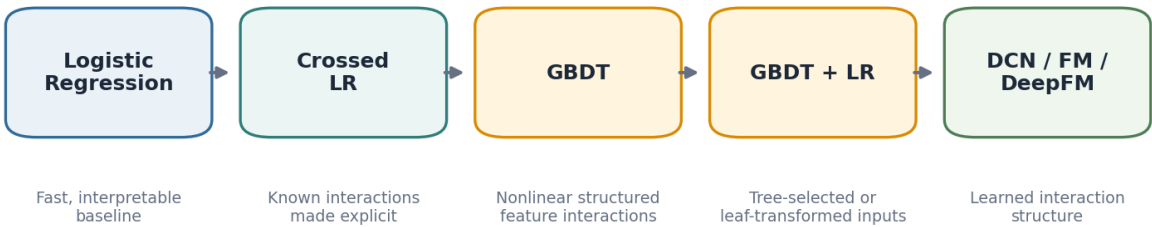
Primary offline metric: _____
Normalized CE baseline: _____
Calibration check: _____
Online success metric: _____
User guardrail: _____

5. Easy-to-implement and structured baselines

Start with a model that is easy to debug, then add only the interaction capacity the data requires.

A practical model progression

The options represent increasing interaction capacity, not a mandatory sequence.



Starting experiment

Use Logistic Regression as the anchor, then compare DCN and DeepFM when automatic feature interaction learning is valuable.

Figure 2.5 - Practical model progression.

Approach	Best when	Main benefit	Main cost
Logistic Regression	A credible baseline is needed quickly	Fast, interpretable, easy to update	Manual interaction engineering
Feature crossing + LR	A few important interactions are already known	Keeps the final model simple	Sparse crosses and domain effort
GBDT	Structured features need nonlinear thresholds	Strong interaction discovery	Sparse IDs and continual updates can be awkward
GBDT + LR	Tree structure should feed a linear scorer	Flexible feature-selection or leaf-ID route	Two-stage pipeline complexity

FROM THE ORIGINAL NOTES

Two distinct GBDT + LR routes

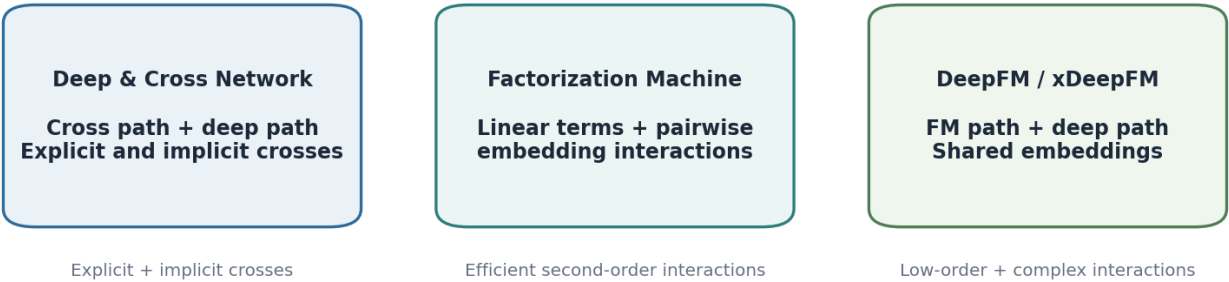
One route uses importance or SHAP to select original features for LR. The other transforms examples into tree-leaf IDs and trains LR on the resulting representation. They remain alternatives rather than one mandatory procedure.

6. Richer interaction models

DCN, FM, and DeepFM differ mainly in how they represent interactions between sparse IDs, dense context, and content features.

How richer models learn interactions

Each family encodes user-ad relationships differently.



Selection question

Which interaction patterns matter, how much interpretability is needed, and what serving latency is acceptable?

Figure 2.6 - Interaction-learning alternatives.

Model	Interaction structure	Best fit
Deep & Cross Network	Explicit cross layers plus a deep network	Known and learned crosses are both valuable
Factorization Machine	Linear terms plus pairwise embedding interactions	Efficient second-order interactions dominate
DeepFM	FM path plus deep nonlinear path with shared embeddings	Low-order and complex interactions both matter
xDeepFM	Adds more explicit higher-order interaction learning	Higher-order structure provides measurable lift

STARTING RECOMMENDATION

Experiment rather than declare a universal winner

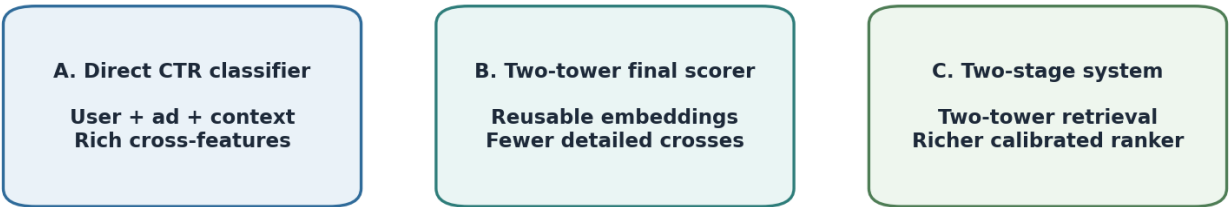
Use Logistic Regression as the anchor and compare DCN and DeepFM. Promote a richer model only when offline probability quality, calibration, latency, and online metrics justify the additional complexity.

7. Two-tower placement and candidate generation

The central system question is whether the two-tower model should perform final scoring or only reduce a large ad inventory to a manageable candidate set.

Where should a two-tower model sit?

Retrieval and final CTR ranking may need different representations.



Starting recommendation

Use the two-tower model for scalable candidate retrieval and preserve a richer classifier for calibrated final ranking.

Figure 2.7 - Three placements for a two-tower model.

CONFIRMED INTERPRETATION

Retrieval and final scoring solve different constraints

A reusable embedding model is attractive for large-scale retrieval, while the final CTR model may need detailed user-ad-context interactions and calibrated probabilities.

Candidate generation and final ranking

Candidate generation and final ranking

Candidate sources may be combined, then deduplicated and enriched before CTR scoring.

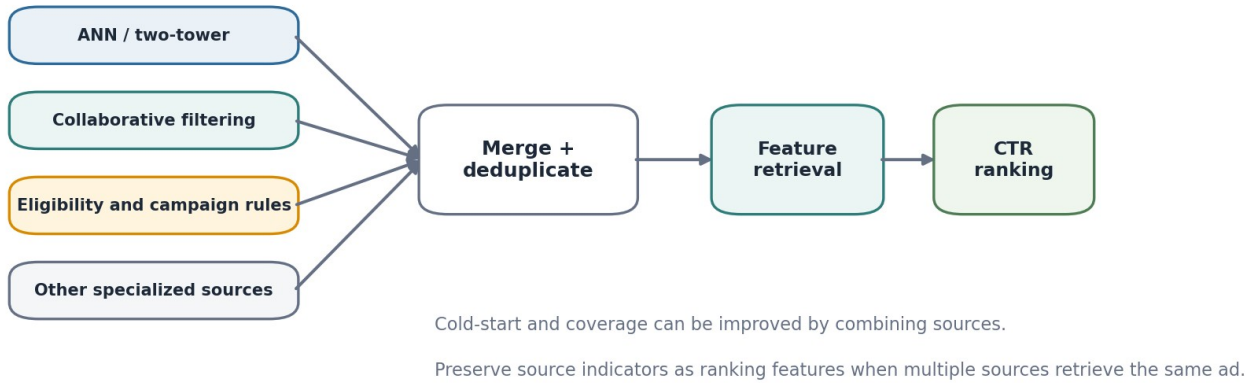


Figure 2.8 - Candidate sources, merge, feature hydration, and final CTR ranking.

STARTING RECOMMENDATION

Use a two-stage system at large inventory scale

Retrieve candidates with reusable user and ad representations, merge and deduplicate multiple sources, then apply a richer calibrated CTR classifier that preserves cross-features.

8. Continual learning and drift response

User interests, active campaigns, inventory, and pricing conditions change. Updating should be triggered by observed drift or quality loss, not added automatically.

Continual learning is a drift response

Choose the smallest update strategy that addresses observed drift without unnecessary forgetting.



Trigger examples

CTR calibration drift, campaign turnover, changing user interests, feature distribution shift, or declining online metrics.

These methods may be used independently or combined; the notes do not prescribe one universal strategy.

Figure 2.9 - Continual-learning alternatives.

Method	Benefit	Risk
Regularization	Preserves important weights while adapting	May restrict useful changes
Replay	Retains older behaviors and reduces forgetting	Storage cost and distorted time distribution
Adapters or progressive capacity	Isolates updates from the core network	Growing model and operational complexity
Combination	Addresses several failure modes	More tuning and attribution difficulty

OPEN DESIGN DISCUSSION

What should trigger an update?

Primary drift signal: _____
Selected update method: _____
Retention risk: _____
Retraining threshold: _____
Rollback criterion: _____

9. Production validation and probability calibration

Shadow deployment validates operations; canary and A/B testing provide live behavior. Calibration determines whether scores can be used as trustworthy probabilities.

Validation, rollout, and probability calibration

Shadow, canary, and A/B testing provide different evidence.

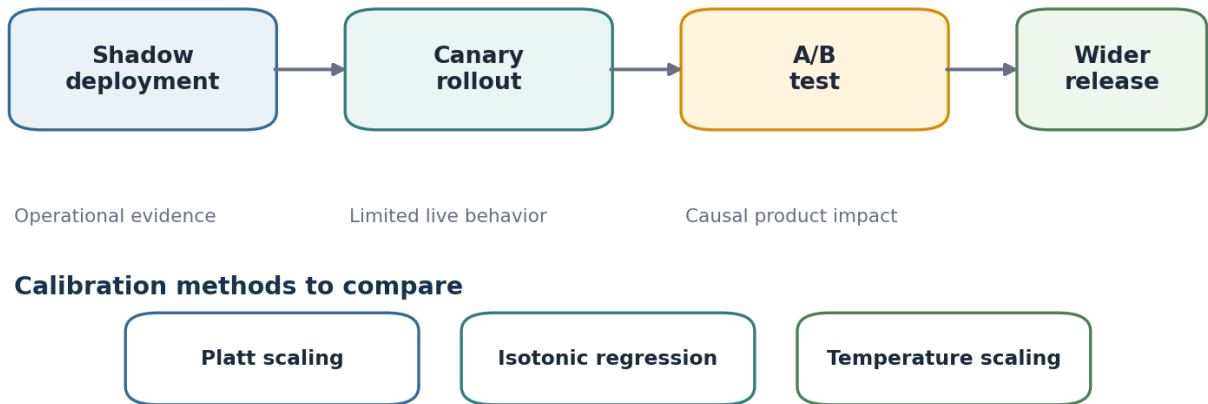


Figure 2.10 - Rollout evidence and calibration choices.

Method	What it provides	What it cannot prove alone
Shadow deployment	Latency, errors, stability, score distribution	Actual click or revenue impact
Canary rollout	Limited live user and business behavior	Full-scale reliability
A/B test	Causal product and revenue comparison	Safe infrastructure ramp by itself

Calibration method	Strength	Constraint
Platt scaling	Simple parametric mapping	Assumes logistic shape
Isotonic regression	Flexible monotonic mapping	Needs enough calibration data
Temperature scaling	Simple score-sharpness adjustment	May be too limited for arbitrary miscalibration

10. Time-aware training and leakage prevention

Features must reflect only information available at request time, and evaluation splits must preserve temporal order.

Prevent future-data leakage

Every feature and label must be computed relative to a prediction-time cutoff.

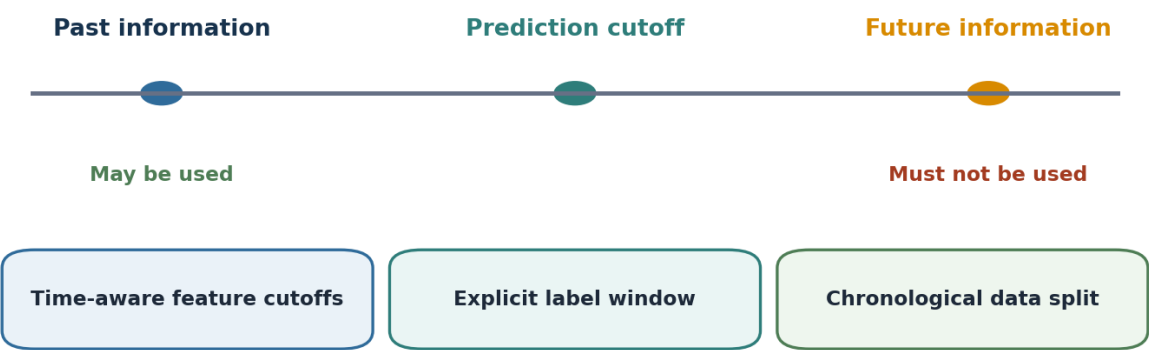


Figure 2.11 - Prediction cutoff and leakage controls.

Leakage route	Example	Control
Aggregates beyond cutoff	Click count includes events after the impression	Point-in-time feature computation
Future campaign outcomes	Final conversion or budget result used as input	Restrict feature availability by timestamp
Post-click information	Conversion or dwell known only after scoring	Define an explicit label window
Random split across time	Later behavior appears in training for earlier evaluation	Use chronological train, validation, and test ranges

OPEN DESIGN DISCUSSION

Leakage audit

Feature timestamp: _____
Prediction cutoff: _____
Label window: _____
Train range: _____
Validation and test ranges: _____

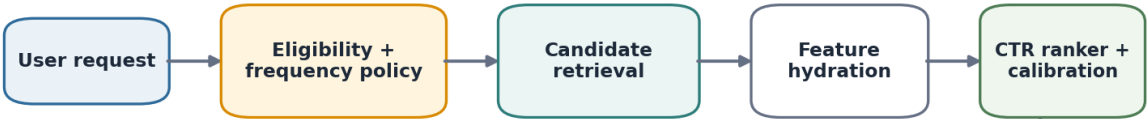
11. Reference architecture

One defensible system combines policy filtering, scalable retrieval, rich final ranking, calibration, and point-in-time learning.

Reference architecture

One reasonable production-balanced design under the current assumptions.

ONLINE SERVING



OFFLINE LEARNING AND VALIDATION



Starting recommendation: two-tower or ANN retrieval for scale, a richer calibrated CTR model for final ranking, and explicit frequency and hide/block guardrails.

Figure 2.12 - Production-balanced reference architecture.

ONE REASONABLE DESIGN

Current reference architecture

Apply eligibility and frequency policy first. Retrieve candidates through two-tower or ANN and other sources, merge and hydrate features, score with a richer CTR model, and calibrate probabilities. Train from point-in-time event logs, evaluate chronologically, and deploy through staged validation.

OPEN DESIGN DISCUSSION

What would change this architecture?

- Inventory-size threshold: _____
- Latency threshold: _____
- Cross-feature lift requirement: _____
- Calibration failure condition: _____
- Simpler acceptable design: _____

12. Decision summary

The table records a starting choice and the condition that should cause the design to change.

Design area	Starting choice	Switch when
Repeated exposure	Explicit frequency or fatigue policy	Data shows no fatigue or a dynamic rule is necessary
Negative labels	Transparent impression-based baseline	Hidden positives materially limit quality
Offline metrics	Cross-entropy, normalized CE, AUC, calibration	A metric fails to predict online outcomes
Baseline	Logistic Regression	A still simpler prior is sufficient
Richer model	Compare DCN and DeepFM	Interaction lift does not justify latency or complexity
Two-tower placement	Candidate retrieval	Inventory is small or cross-feature loss is unacceptable
Final scorer	Richer calibrated CTR classifier	Two-tower similarity performs equivalently
Continual learning	Monitor and retrain when triggered	Stable behavior makes scheduled retraining sufficient
Rollout	Shadow, then limited live validation and A/B test	Risk profile requires another ordering
Leakage control	Point-in-time features and chronological splits	Never optional

INTERVIEW-READY ANSWER

Trace every component to a product or system need

I would begin with Logistic Regression, explicit label construction, normalized cross-entropy against a named baseline, and frequency guardrails. At large inventory scale, I would retrieve with a two-tower or ANN system and use a richer calibrated CTR model for final ranking. I would validate time-aware offline metrics, shadow operations, limited live traffic, A/B impact, and leakage controls before full release.

Attribution

This project is independent study work and is not affiliated with the book authors or publisher. It is not a substitute for the original book. Public versions use original wording and original diagrams and exclude handwritten source scans.